

Jane + Ads — Platform Integration Guide (Google Ads / TikTok Ads)

Audience: developers integrating a second and third ad platform into the existing Jane + Ads system.

Package: `app/agents/jane_ads/` in `uri-social-backend`.

Status today: Meta is the only platform with a real, live adapter. Google and TikTok are fully modeled in the decision logic (Jane already reasons about when she'd pick them) but have no adapter implementation — every campaign is currently force-routed onto Meta regardless of what Jane decides. That's the seam this doc explains how to close.

All file:line references below point at `origin/develop` as of 2026-08. Paths are relative to `app/agents/jane_ads/` unless stated otherwise.

1. The one-paragraph mental model

Jane + Ads is a **conversation → decision → creative → platform** pipeline. A client describes what they want in plain English; a natural-language layer extracts a structured `CampaignRequest`; a pure, deterministic decision engine decides which platform(s), budget split, and duration; a creative layer writes copy and generates/collects an image or video; and finally a **platform adapter** turns that finished plan into a real, paused ad on the actual platform. Everything up to the adapter is already fully built and platform-agnostic — nothing in the conversation, decision, creative, wallet, or Plan Defence layers needs to change for Google or TikTok. **The adapter is the whole job.**

...

User message



`jane_consultant.py / nl.py` (LLM extraction → `CampaignRequest` — no platform field)



`decision_engine.py` (pure function → `CampaignPlan` with platform(s) already chosen)



`creative.py` (writes copy + image/video → `AdCreative` — platform-agnostic)



`wallet.py / caps.py` (spend authorization — platform-agnostic)



****Read `adapters/meta.py` end to end before writing anything.**** It's the only real implementation and the pattern to copy — not because Meta's specific API calls matter to you, but because the *shape* of a correct adapter (pre-flight validation, sequential resource creation, PAUSED by default, persisting platform IDs to Mongo, delta-based conversation polling) is the same regardless of platform.

3. What Meta's adapter actually does (your reference implementation)

``MetaAdPlatformAdapter` (`adapters/meta.py:127`):`

- ****Auth****: constructor takes ``db`` + an ad account ID + access token (falls back to ``settings.META_AD_ACCOUNT_ID`/`settings.META_SYSTEM_TOKEN``), raises ``MetaAPIError`` if either is missing (meta.py:132-145). Every Graph call passes ``access_token`` as a query param, not a bearer header — match whatever your platform's SDK/API actually expects, this isn't a convention to preserve.

- ****`launch\_campaign`**** (meta.py:192-373) — sequential API calls in one ``httpx.AsyncClient``:

1. Pre-flight: require ``plan.page_id``, ``plan.creative.image_url``, and (unless goal is FOLLOWERS) ``plan.whatsapp_number``. ****Fail loudly before creating anything**** if a precondition is missing — don't create a half-built campaign.

2. If the creative is a video, upload it first and poll until Meta reports it ready (up to 20×5s) before referencing it anywhere else.

3. Create the campaign object — ``status: "PAUSED"``.

4. Create the ad set — daily budget, targeting, optimization goal. Meta branches optimization goal on whether the goal is FOLLOWERS (``POST_ENGAGEMENT``) or everything else (``CONVERSATIONS`` + ``destination_type: "WHATSAPP"`` + ``promoted_object: {page_id, whatsapp_phone_number}``). ****This exact click-to-WhatsApp mechanic is Meta-specific**** — Google Ads and TikTok have their own equivalent conversion/destination concepts; map to whichever of theirs is closest to "a click that starts a WhatsApp conversation," or to their own native lead/conversion action if there's no WhatsApp equivalent (see §6).

5. Create the ad creative object (copy + image/video reference).

6. Create the ad itself, referencing the creative — ``status: "PAUSED"``.

7. ****Persist `{campaign\_id, adset\_id/equivalent, ad\_id, business\_id, platform, last\_conversation\_count: 0}` to a Mongo collection**** (``jane_ads_meta_campaigns`` for Meta). Your adapter needs its own equivalent collection (e.g. ``jane_ads_google_campaigns``) — this is how ``fetch_per_ad_spend``/``poll_conversations``/``billing`` find your campaigns later, and how the campaign-list endpoints resolve which adapter to call (see §7).

****Every campaign is created PAUSED, never ACTIVE**** (meta.py:24-26) — this is a hard product rule, not a Meta quirk. A client reviews and explicitly activates in their own Ads Manager (or via ``/status``). Your adapter must default to whatever your platform's equivalent of paused/draft/inactive is.

- ****`fetch\_per\_ad\_spend`**** (meta.py:383-417) — returns ****cumulative**** spend per ad (not a delta) via an insights/reporting call.

- ****`poll\_conversations`**** (meta.py:503-556) — ****no live webhook today****; Meta's real webhook delivery is explicitly flagged as a separate, not-yet-built item (meta.py:27-31). Instead it polls an

insights endpoint and computes the `delta` since `last_conversation_count` stored per-campaign in Mongo, specifically to avoid double-charging the wallet for the same conversation twice. If your platform has a real webhook, prefer it — but whatever the mechanism, the delta-not-total discipline is required, since `billing.py` charges off what this method returns.

- `pause_ad` (meta.py:558-567) — a single status-flip call.

Adapter-only extras beyond the base interface that Meta's adapter also has, used elsewhere in the router (not part of the required interface, but expect to want equivalents): `get_delivery_estimate` (reach estimate, feeds `summary.py`'s Campaign Summary), `fetch_campaign_summary` (combined snapshot for the campaign list view), `set_delivery` (cascades active/paused down the object hierarchy), `delete_campaign`.

4. What's already done — you should not need to touch these

Everything below reasons purely from `CampaignRequest`/`CampaignPlan`/`CampaignSummary` and makes `no platform API calls`. Confirmed platform-agnostic by direct inspection, not assumption:

- `jane_consultant.py` / `nl.py` — the LLM-based conversation layer that turns a plain-English message into a `CampaignRequest`. There is no platform field anywhere in this extraction — platform is a `decision`, never something the user is asked for directly. The consultant's system prompt does reference the real per-platform economics (Meta's █1,610 daily floor, Google's no-floor-but-needs-clicks, TikTok's █31,000 floor + video requirement — `jane_consultant.py:211-216`) purely as informational context for the model's own reasoning about behaviour/budget framing, not as something it writes into the request.

- `decision_engine.py` — pure, deterministic, already fully models all three platforms (\$5 below has the exact logic). This is Jane's actual reasoning about `when` she'd pick Google or TikTok — it already works correctly today for those platforms in isolation; only the launch step ignores its answer.

- `creative.py` — all four creative sources (GENERATE/UPLOAD/DRAFT/RECOMPOSITE) produce a platform-agnostic `AdCreative`. One caveat: the generated image is currently always requested in a fixed Meta-Stories-shaped spec (`_AD_IMAGE_PLATFORM="instagram"`, `_AD_IMAGE_TYPE="story"`, `creative.py:52-53`) — vertical 9:16. If your platform needs a different aspect ratio or has its own creative-spec requirements (TikTok is video-only per `CreativeKind.NONE`/`TIKTOK_REQUIRES_VIDEO` — Google Search often needs no creative at all), that's a real gap to account for, not a false assumption to inherit.

- `wallet.py` / `store.py` / `caps.py` — the money system. `WalletService.authorization_for` and `SpendAuthorization` carry no platform field at all; the wallet is per-business and shared across whatever platform(s) a campaign runs on. Two-layer cap enforcement (per-business in `caps.py:47-62`, per-account pool in `caps.py:72-100`) is likewise platform-agnostic. Nothing here changes for a new platform.

- `summary.py` (Campaign Summary — "here's my thinking") and `plan_defence.py` (the "why this budget?" / what-if / challenge Q&A layer) — both reason only from already-decided `CampaignPlan`/`CampaignSummary` objects, never call a platform API directly. `summary.py`'s only live data input is an optional `delivery_estimate` dict the `caller` fetches (today, only ever from Meta's adapter) — if you want reach estimates to show up correctly for a Google/TikTok-routed plan, fetch it from your adapter's own reach-estimate equivalent and pass it in the same way; the summary logic itself needs no changes.

****Practically: do not touch these files.**** If you find yourself wanting to, it likely means the seam you actually need is in router.py (§7) or a new platform-specific connection module (§6).

5. What the decision engine already knows about your platform

`decision\_engine.py` (layer order: GOAL → BEHAVIOUR → BUDGET → CREATIVE gate → GEOGRAPHY → RECOMMEND+EXPLAIN, decision_engine.py:8-15) already has real logic for Google and TikTok — you're not adding platform-selection logic, only implementing what happens once one is selected.

****Behaviour → platform lean**** (`\_LEAN`, decision_engine.py:92-96):

```
```python
SEARCH: [Platform.GOOGLE]
DISCOVER: [Platform.META, Platform.TIKTOK]
MIXED: [Platform.META, Platform.GOOGLE]
```
```

`PurchaseBehaviour` is resolved from a business-type keyword default, overridable by what the client actually stated, then adjusted by goal implications (decision_engine.py:67-88) — this is the actual "does Jane recommend Google here" logic, already working.

****TikTok's hard creative gate**** (decision_engine.py:158-161) — applied **before** affordability, so TikTok can never be chosen without video: `if Platform.TIKTOK in lean and not request.creative.has\_video: lean.remove(Platform.TIKTOK)`.

****Useful-minimum budget floors**** (`constants.py:19-23`), used to decide whether a platform is even affordable:

```
```python
USEFUL_MIN_NGN = {"meta": 5_000.0, "google": 5_000.0, "tiktok": 50_000.0}
```
```

And the (currently informational-only) hard daily floors (`constants.py:27-31`):

```
```python
HARD_FLOOR_DAILY_NGN = {"meta": 1_610.0, "google": 0.0, "tiktok": 31_000.0}
```
```

`TIKTOK\_REQUIRES\_VIDEO: bool = True` (constants.py:34). These numbers came from the PRD's Part A2 — confirm current values against each platform's live docs before you rely on them for real budget gating; they're explicitly flagged in the file as "verified 2026" and meant to be the **only** place platform economics live, never hard-coded elsewhere.

****Multi-platform budget split**** (decision_engine.py:184-201): if the budget can't fund the combined useful-minimums of every affordable platform in the lean set, Jane concentrates the whole budget on the single best fit rather than starving several; otherwise it splits evenly. ****A campaign plan can legitimately contain more than one `PlatformPlan`**** (e.g. Meta + Google under `MIXED` behaviour) — your adapter and the router's launch logic need to handle a plan where only **some** of the platforms are yours to launch.

None of this changes for your integration. The only thing missing today is a launch-time consumer that respects what this engine already decided.

6. What you actually need to build, per platform

1. `**`adapters/google.py` / `adapters/tiktok.py`**` implementing ``AdPlatformAdapter``'s four methods, following the shape in §3. Own Mongo collection for persisted campaign records (mirroring ``jane_ads_meta_campaigns``).

2. `**A connection/auth module**`, mirroring ``ads_connection.py`` but for your platform's OAuth. This is a real gap, not reusable machinery — ``ads_connection.py`` is explicitly Meta/Facebook-specific (reads ``social_connections`` filtered to ``platform: "facebook_ads"``, verifies Facebook Graph scopes). Google Ads needs Google OAuth + an Ads account link; TikTok needs TikTok Business OAuth. The `*pattern*` worth copying: an explicit ``ConnectionState`` enum (never a single boolean — ``ads_connection.py:44-51`` has ``NONE, CONTENT_ONLY, ADS_NO_WHATSAPP, READY, EXPIRED, NO_PAGE``; yours will have different states but the same "six explicit states, one resolve function, one typed exception" shape) plus a ``resolve_*_page_for_launch``-equivalent pre-flight gate that's checked `**before a campaign is built, not at launch**` — a client should never be walked through a whole planning conversation only to hit a connection wall at the end.

3. `**Wire into `router.py`**` This is the actual integration point. Today, ``router.py:1129-1137`` (inside ``/meta/plan-from-message``'s build path) `**unconditionally force-routes every plan onto Meta**`:

```
```python
For now, always launch on Meta — it's the only platform with a live adapter
(Google/TikTok are still pending, #7/#8). If Jane's decision landed elsewhere,
force the plan onto Meta so the demo always produces a real ad. Jane's original
recommendation is still surfaced in the response for transparency.
jane_platforms = [p.platform.value for p in plan.platforms]
forced_to_meta = not any(p.platform == Platform.META for p in plan.platforms)
if forced_to_meta:
 plan = apply_platform_override(plan, [Platform.META])
else:
 plan.platforms = [p for p in plan.platforms if p.platform == Platform.META]
```
```

This needs to become a real dispatch: for each ``PlatformPlan`` in ``plan.platforms``, resolve the matching adapter (Meta/Google/TikTok) and connection state, and either launch on all of them or degrade gracefully (existing ``forced_to_meta`/`jane_platforms`` fields in the response already exist specifically to tell the client "Jane wanted X, here's what actually happened" — extend that pattern, don't remove it).

The router also imports ``MetaAdPlatformAdapter`` directly in a few other places that assume Meta: ``GET /meta/campaigns`` (`router.py:1802-1803``), ``POST /meta/campaigns/{id}/status``, ``DELETE /meta/campaigns/{id}`` (`router.py:1880+, 1909+``). Each needs to become adapter-aware (likely: store ``platform`` on the persisted campaign record, look it up, and dispatch to the matching adapter).

4. `**`billing.py`'s `reconcile_ad_spend_charges`**` (`billing.py:50-168``) is currently hardcoded to Meta — it imports ``MetaAdPlatformAdapter`` directly (`billing.py:58``) and reads the

``jane_ads_meta_campaigns`` collection by name. This recoups real platform spend × ``AD_SPEND_MARKUP`` (1.10, ``constants.py:56``) from the customer's wallet and pauses campaigns that run dry — it needs a per-platform equivalent (or a platform-dispatching rewrite) or Google/TikTok spend will never get billed back.

5. **Creative-spec handling**, per §4's caveat — decide whether your platform can consume the same Meta-Stories-shaped generated image as-is, needs a different aspect ratio requested from ``creative.py``'s image generation, or (Google Search, ``CreativeKind.NONE``) needs no creative asset at all.

7. Naming/endpoint convention to follow

Every endpoint under this router is prefixed ``/jane-ads`` (``router.py:41``). The Meta-specific ones are grouped under ``/jane-ads/meta/*`` (``plan-from-message``, ``launch-from-message``, ``plan/{id}/launch``, ``plan/{id}/ask``, ``campaigns``, etc.) precisely because they assume one platform. When you wire in a real second platform, the honest options are: (a) genuinely generalize these into platform-agnostic paths once dispatch exists, or (b) keep platform-specific paths (``/jane-ads/google/...``) if the request/response shapes end up meaningfully different per platform. Don't silently repurpose the ``/meta/*`` paths to secretly mean "any platform" — that's exactly the kind of implicit behavior this whole system otherwise goes out of its way to avoid (see ``plan.trace``, ``forced_to_meta``, ``jane_recommended_platforms`` — everything here is designed to make Jane's actual reasoning and any override visible to the client, never silent).

8. Testing conventions already established

- ``adapters/mock.py`` is a deterministic, no-network simulation of a platform adapter — the whole decision-engine → wallet → caps → conversation-flow pipeline runs and is tested end-to-end against it without hitting any real platform. Useful as a second reference for "what's the minimal correct shape of an adapter" alongside Meta's real one.

- Tests live in ``tests/test_jane_ads_*.py`` at the repo root, one file per module roughly mirroring the package layout (``test_jane_ads_decision_engine.py``, ``test_jane_ads_meta_adapter.py``, ``test_jane_ads_wallet.py``, etc.) — follow that convention for ``test_jane_ads_google_adapter.py``/``test_jane_ads_tiktok_adapter.py``.

- Copy ``.env`` into any fresh worktree before running pytest — ``pydantic`` Settings validation fails loudly (6+ missing-field errors) without it.

- Compile-check (``python -m py_compile``) after every meaningful edit, then the targeted test file, then the full ``pytest tests/ -k jane_ads -q`` before considering a change done.

9. One live caveat as of this writing

``ads_connection.py``'s ``REQUIRED_ADS_SCOPES`` set (``ads_connection.py:31-38``) currently has ``ads_management`` temporarily removed as a staging-only debugging workaround (dated 2026-08-03, explicitly flagged in-code as **must be reverted before reaching prod**). This is unrelated to the Google/TikTok integration itself, but if you're reading ``ads_connection.py`` as a reference pattern for your own connection module, don't copy that specific line — it's a temporary

hole, not the intended permanent scope set.

Suggested order of work

1. Read ``adapters/base.py``, ``adapters/meta.py``, and ``adapters/mock.py`` in full before writing code.
2. Build your adapter against ``adapters/mock.py``'s test harness first — prove ``launch_campaign``/``fetch_per_ad_spend``/``poll_conversations``/``pause_ad`` work correctly in isolation before touching ``router.py`` at all.
3. Build the connection/OAuth module (§6.2), mirroring ``ads_connection.py``'s state-machine shape.
4. Only then touch ``router.py``'s dispatch logic (§6.3) — this is the highest-blast-radius change (it's the current Meta demo path everyone already depends on), so it should be the last piece, done carefully, with the existing Meta flow's tests still green throughout.
5. ``billing.py`` reconciliation (§6.4) can happen in parallel once your adapter persists campaign records in its own collection.